

Task ##P/C – Spike: Core 1

Goals:

The goal is to develop a one-activity game for android that makes use of layouts and localisation.

Summary:

A simple dice game which incorporates a single activity window with buttons, image, and text. Users can roll a dice which will generate a random number between 1 to 6 and then they must decide to add or subtract that number to the score. The aim of the game is to reach the score 20. The app should have a separate layout for portrait and landscape mode, while preserving the app states between orientation change. Added support for Bengali language will also be added.

Knowledge gained:

- Understanding Android app lifecycle.
- Incorporating random dice rolls with the option to use a specific seed for testing.
- Implementing locale-specific number and button representation.
- Implementing different layout based on the screen orientation.
- Implementing state saves between lifecycle change.
- Implementing a secondary language support.

Tools and Resources Used

This section lists related software, tools, libraries, APIs, and other resources used for this knowledge gap.

- Android Studio: The official integrated development environment (IDE) for Google's Android operating system.
- Kotlin Documentation: Documentation on the Kotlin programming language.
- Android Developers Website: Official documentation and tutorials for Android app development.
- Android Material Design Guidelines: Documentation on creating visually compelling Android apps.
- Stack Overflow: Community-driven Q&A site for programming-related topics.

Knowledge Gaps and Solutions

Gap 1: Saving game state between lifecycles.

- **Observation:** the buttons, dice, and score resets when the layout is changed from portrait layout to landscape layout.
- **Steps to address the Gap:**
 - Using '*savedInstanceState*' class to save the widget values to their corresponding variables.
 - Get the saved variables from the '*savedInstanceState*' and set the widget values accordingly.

Gap 2: Using a seed for random number creation.

- **Observation:** the built-in random function does not allow a seed to be used. A seed is required for the test cases/
- **Steps to address the Gap:**
 - Used the *kotlin.random.Random* library to create a class variable with the type random, which allows the generation of random numbers with a seed value
 - Used another variable *rollCount* to count the number of rolls to pass it using the *savedInstance* class and account for random generation after a state change.

Gap 3: Determining button availability.

- **Observation:** over-extensive state saving is required to determine the availability of each button after a lifecycle change.
- **Steps to address the Gap:**
 - Used a single variable 'isRollState' to determine if the game is on a state to roll the dice. Used logic to make the add and sub buttons available when this variable is false.

Open Issues and Recommendations

Issue 1: Hard coded state-load

- **Description:** the saved variables are loaded at the beginning of the onCreate() function, inside a if statement. This is hard coded, and the necessary changes made are also inside this if statement
- **Recommendation:** Following OOP principles, the code inside this if statement can be divided in several functions or maybe a LoadState class.

Issue 2: Complex functions

- **Description:** Some functions execute more tasks than one and they can be confusing and may break the whole app if changed.
- **Recommendation:** Break up complex functions into multiple functions, each with their own task.

Issue 3: Double declarations for button state

- **Description:** Some functions change the button state to the current state.
- **Recommendation:** refactor code and remove any double declarations of the same value for button states, and implement an class for button control.

GitHub link: <https://github.com/SoftDevMobDev-2023-Classrooms/core1-Sal-theSadmanShark>

Description of the Code

State Save Variables:

These variables are used to store the state save values.

```
private const val TAG = "MainActivity" // used for logging
// variables for saveInstanceStates
private const val SCORE_KEY = "score_key"
private const val DICE_KEY = "dice_key"
private const val GAME_STATE_KEY = "game_state_key"
private const val ROLL_COUNT = "roll_count"
```

MainActivity class:

This is the main activity of our android app. This houses all our other code. It has all the private late initialised variables for the widgets. Then the local variables for state saving are created.

```
Salim Sadman
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    // assign all the android widget variables by ID
    btRoll = findViewById(R.id.btRoll)
    ivDice = findViewById(R.id.ivDice)
    btAdd = findViewById(R.id.btAdd)
    btSub = findViewById(R.id.btSub)
    tvScore = findViewById(R.id.tvScore)
    btReset = findViewById(R.id.btReset)
```

It has an If condition that checks if there are any saved Instance variables for loading data from a previous session. If there are any saved variables from a previous cycle, then change the wedged values accordingly.

```
// Check if there's a saved state
if (savedInstanceState != null) {
    currentScore = savedInstanceState.getInt(SCORE_KEY, defaultValue: 0)
    diceNum = savedInstanceState.getInt(DICE_KEY, defaultValue: 0)
    isRollState = savedInstanceState.getBoolean(GAME_STATE_KEY, defaultValue: true)
    rollCount = savedInstanceState.getInt(ROLL_COUNT, defaultValue: 0)

    for (i in 1..rollCount) {
        random.nextInt( from: 1, until: 7)
    }

    setDice()
    tvScore.text = currentScore.toString()

    // check if the game is in the roll state or arr/sub state
    btRoll.isEnabled = isRollState
    canAddSub(!isRollState)

    if (currentScore > 20) {
        tvScore.setTextColor(Color.RED)
    } else if (currentScore == 20) {
        tvScore.setTextColor(Color.GREEN)
        canRoll( b: false)
        canAddSub( b: false)
        Log.i(TAG, msg: "gameOver") // Log the game over even
    } else {
        tvScore.setTextColor(Color.BLACK)
    }
}
}
```

Then there are onclick listeners for all the buttons which execute different tasks depending on the button pressed. The roll button rolls the dice, shows an icon according to the dice number and unlocks the add, sub buttons, while locking the roll button. The add button adds the dice number to the score and makes the roll button available. The subtract button subtracts the dice number from the score and makes the roll button available. The reset button resets the score.

```
// roll button action
btRoll.setOnClickListener { it: View!
    rollDice()
    setDice()
    canAddSub( b: true)
    canRoll( b: false)
}

// Add button action
btAdd.setOnClickListener { it: View!
    currentScore += diceNum
    updateScore()
    Log.i(TAG, msg: "numAdded") // Log add event
}

// Subtract button action
btSub.setOnClickListener { it: View!
    currentScore -= diceNum
    updateScore()
    Log.i(TAG, msg: "numSubtracted") // Log subtract event
}

// reset button action
btReset.setOnClickListener { it: View!
    currentScore = 0
    updateScore()
    ivDice.setImageResource(R.drawable.zero)
    diceNum = 0
    Log.i(TAG, msg: "scoreReset") // Log the reset even
}
```

UpdateScore function:

This function handles the button availability and updates the score. It also has some if conditions to check the score and apply colour to the text. If the score goes below zero it sets the score to zero.

```
// update the score with the current score and disable add, sub button. then enable roll
± Salim Sadman
private fun updateScore() {
    tvScore.text = currentScore.toString()
    canAddSub( b: false)
    canRoll( b: true)

    if (currentScore > 20) {
        tvScore.setTextColor(Color.RED)
    } else if (currentScore == 20) {
        tvScore.setTextColor(Color.GREEN)
        canRoll( b: false)
        Log.i(TAG, msg: "gameOver") // Log the game over even
    } else if (currentScore < 0) {
        currentScore = 0
        tvScore.text = currentScore.toString()
    } else {
        tvScore.setTextColor(Color.BLACK)
    }
}
```

canRoll, canAddSub functions:

These functions have the same tasks, they take a Boolean value and according to the value, makes the buttons enabled or disabled.

```
// make the roll button clickable or hidden
± Salim Sadman
private fun canRoll(b: Boolean) {
    btRoll.isEnabled = b
    isRollState = b
}

// make the add, sub buttons clickable or hidden
± Salim Sadman
private fun canAddSub(b: Boolean) {
    btAdd.isEnabled = b
    btSub.isEnabled = b
}
```

rollDice function:

this function uses the random class variable and generates a random number from the range 1 to 6

setDice function:

this function uses the diceNum class variable and sets the dice icon to the appropriate dice image.

```
Salim Sadman
private fun setDice() {
    when (diceNum) {
        1 -> {
            // change dice icon to dice1
            ivDice.setImageResource(R.drawable.dice1)
        }

        2 -> {
            // change dice icon to dice2
            ivDice.setImageResource(R.drawable.dice2)
        }

        3 -> {
            // change dice icon to dice3
            ivDice.setImageResource(R.drawable.dice3)
        }

        4 -> {
            // change dice icon to dice4
            ivDice.setImageResource(R.drawable.dice4)
        }

        5 -> {
            // change dice icon to dice5
            ivDice.setImageResource(R.drawable.dice5)
        }

        6 -> {
            // change dice icon to dice6
            ivDice.setImageResource(R.drawable.dice6)
        }

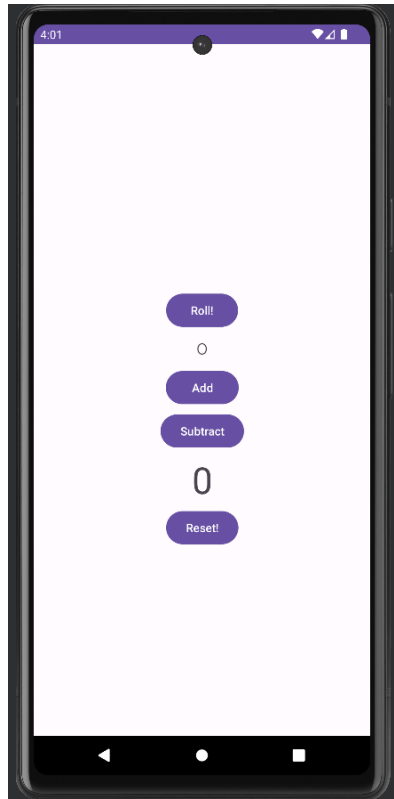
        else -> {
            ivDice.setImageResource(R.drawable.zero)
        }
    }
}
```

onSaveInstanceState function:

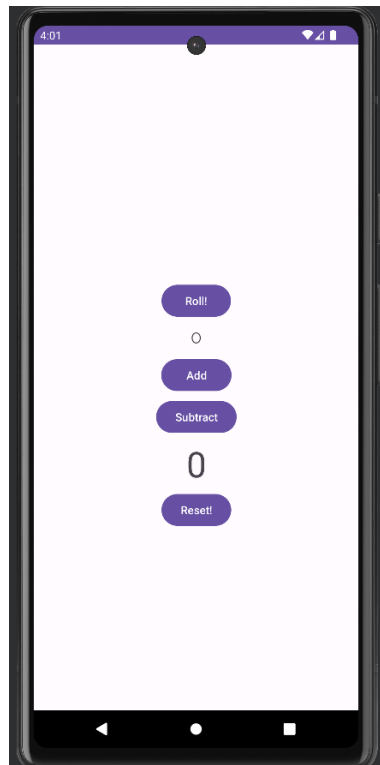
here the state saving variables are passed to the outState.

Demonstration

After starting the app, we see the main layout. It has four buttons, one Image for the dice and a text for score.



Users can click the roll button to roll a dice. Add button will and subtract button will subtract the score, while reset will reset the score.



If the screen orientation is changed, the layout of the app changes.



The app also has Bengali language support.



Test Cases

The screenshot shows the Android Studio interface during a test run. The 'Run' tab is active, displaying the test results for 'MainActivity2Test'. The test results are as follows:

Test	Duration	Pixel_6a_API_33
✓ MainActivity2Test	17 s	6/6
✓ testResetButton	17 s	6/6
✓ clickRollAddSubtractButton3Times	3 s	✓
✓ testLowerLimitsOfScore	2 s	✓
✓ clickRollAddButton3Times	2 s	✓
✓ testScoreOnRotation	2 s	✓
✓ testScoreOnRotationWithClick	2 s	✓

The status bar at the bottom indicates 'Tests Passed 6 passed' and 'Gradle build finished in 30 s 26 ms (4 minutes ago)'. The right sidebar shows the 'Test Results' tab with a summary of the test run, including the task ':app:connectedDebugAndroidTest' and the build status 'BUILD SUCCESSFUL in 29s'.